Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

RAPORT

la Lucrarea de laborator Nr. 8

Disciplina: Tehnici de programare

Tema: „Colecții: liste, tupluri, mulțimi, dicționare”

A efectuat: st. gr. SI-264 Danuța Daniel

A verificat: asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Scopul lucrării de laborator constă în studierea și aplicarea colecțiilor din limbajul Python liste, tupluri, mulțimi și dicționare prin realizarea unei unelte de enumerare a configurațiilor de securitate Linux.

În cadrul lucrării au fost prelucrate capturi referitoare la fișierele SUID, drepturile sudo și sarcinile cron, au fost identificate constatările cu risc ridicat și a fost generat un raport JSON. Ulterior, configurația actuală a fost comparată cu una de referință folosind operații cu mulțimi, iar diferența a fost verificată printr-un test automat cu pytest.

MERSUL LUCRĂRII

1. Structura proiectului

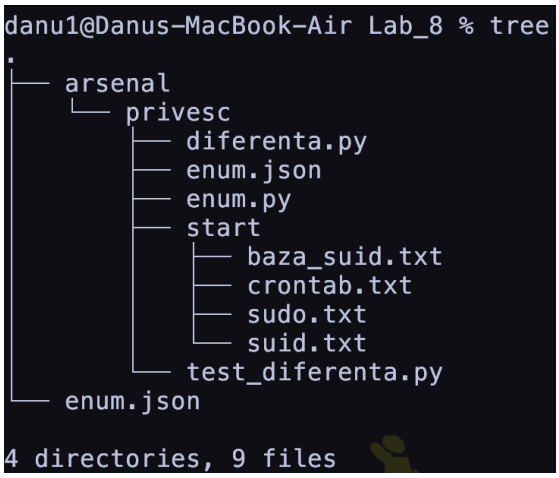


Fig. 1.1 – Structura proiectului

A1–A2.

```python
import argparse, json, os


# A1

def citeste_linii(cale):
    ''' Intoarce lista liniilor .strip(), fara cele goale si fara cele cu # '''
    with open(cale, encoding="utf-8") as f:
        return [
            linie.strip()
            for linie in f
            if linie.strip() and not linie.strip().startswith("#")
        ]

# cai = citeste_linii("arsenal/privesc/start/suid.txt")
# print("linii SUID:", len(cai))


# A2

def nume(cale):
    ''' intoarce ce e dupa ultimul "/" '''
    return cale.rsplit("/" , 1)[-1]

# nume_unice = {nume(c) for c in cai}
# print("SUID distincte:", len(nume_unice))

```

Fig. 2.1 – A1/A2: citirea datelor și numele unice

A3.

```python
    # A3

    GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
            "less", "awk", "python3", "perl"}

    def suid_constatari(cale):
        cai = citeste_linii(cale)
        rezultat = []
        ''' for every unique path , in sorted order , dict cale/risc '''

        for c in sorted(set(cai)):
            rezultat.append({
                "cale": c,
                "nume": nume(c),
                "risc": "mare" if nume(c) in GTFO else "normal"
            })

        return rezultat


    suid = suid_constatari("arsenal/privesc/start/suid.txt")
    mari = [c for c in suid if c["risc"] == "mare"]
    # print(len(suid), "SUID,", len(mari), "cu risc mare")
    # print(sorted(c["nume"] for c in mari))
```

Fig. 2.2 – A3: lista constatărilor SUID

A4.

```
55    # A4
56
57    def sudo_constatari(cale):
58        out = []
59
60        for ln in citeste_linii(cale):
61            user, drept = ln.split(" : ", 1)
62
63            out.append({
64                "user": user.strip(),
65                "drept": drept.strip(),
66                "risc": "mare" if "NOPASSWD" in drept else "normal"
67            })
68
69        return out
70
71
72    def cron_constatari(cale):
73        out = []
74
75        for ln in citeste_linii(cale):
76            campuri = ln.split()
77            user = campuri[5]
78            comanda = " ".join(campuri[6:])
79
80            out.append({
81                "user": user,
82                "comanda": comanda
83            })
84
85        return out
86
87    def enumereaza(d):
88        return {
89            "suid": suid_constatari(os.path.join(d, "suid.txt")),
90            "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
91            "cron": cron_constatari(os.path.join(d, "crontab.txt"))
92        }
93
94
95    # raport = enumereaza("arsenal/privesc/start")
96
97    # for categorie, lista in raport.items():
98    #     print(categorie, len(lista))
99
```

Fig. 2.3 – A4: prelucrarea drepturilor sudo și a sarcinilor cron

A5.

```
101    # A5
102
103    def main():
104        p = argparse.ArgumentParser()
105        p.add_argument("capturi")
106        p.add_argument("--json", action="store_true")
107        a = p.parse_args()
108
109        raport = enumereaza(a.capturi)
110
111        if a.json:
112            print(json.dumps(raport, indent=2, ensure_ascii=False))
113            return
114
115        for categorie, lista in raport.items():
116            mari = [c for c in lista if c.get("risc") == "mare"]
117            print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")
118
119        utilizatori = (
120            {c["user"] for c in raport["sudo"]}
121            | {c["user"] for c in raport["cron"]}
122        )
123
124        with open("enum.json", "w", encoding="utf-8") as f:
125            json.dump(raport, f, indent=2, ensure_ascii=False)
126
127        print("Raport salvat in enum.json")
128
129
130    if __name__ == "__main__":
131        main()
```

Fig. 2.4 – A5: rezultatul executării și raportul JSON

B1.

```
1
2     # A1
3
4     from arsenal.privesc.enum import citeste_linii
5
6     def cai(fisier):
7         return set(citeste_linii(fisier))
8
9     baza = cai("arsenal/privesc/start/baza_suid.txt")
10    acum = cai("arsenal/privesc/start/suid.txt")
11
12    # print("baza:", len(baza), " acum:", len(acum))
13
```

Fig. 3.1 – B1: construirea mulțimilor

B2

```
15    # A2
16
17    noi = acum - baza
18    print(len(noi), "SUID noi fata de sistemul curat")
19    for c in sorted(noi):
20        print(" ", c)
21
```

Fig. 3.2 – B2: diferența dintre mulțimi

B3.

```
23    # A3
24
25    from arsenal.privesc.enum import nume, GTFO
26
27    risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
28    periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
29    print("periculoase NOI:", periculoase)
```

Fig. 3.3 – B3: clasificarea fișierelor noi

B4.

```
32    # A4
33
34    def test_doar_ce_e_nou():
35        acum = {"a", "b", "c"}
36        baza = {"a", "b"}
37        assert acum - baza == {"c"}
```

Fig. 3.4 – B4: rezultatul testului pytest

CONCLUZII

În urma realizării lucrării de laborator nr. 8, am exersat utilizarea listelor, mulțimilor, tuplurilor și dicționarelor în Python pentru citirea, organizarea și analizarea datelor. Am identificat fișiere SUID cu risc mare și am generat un raport în format JSON.

De asemenea, am aplicat diferența dintre mulțimi pentru compararea configurațiilor și am verificat funcționarea acestei operații cu pytest. Lucrarea m-a ajutat să înțeleg mai bine folosirea colecțiilor Python în sarcini practice de analiză a securității.